

Lecture 12: Software Configuration Management (SCM)

1. Introduction to Software Configuration Management (SCM)

What is SCM?

- **Definition:** Software Configuration Management (SCM) refers to the process of managing, tracking, and controlling changes in software during its lifecycle. It ensures that the software is developed, tested, and maintained consistently over time.
- **Purpose of SCM:**
 1. **Track and Control Changes:** SCM allows teams to track every change made to software, enabling them to manage, revert, or merge changes effectively.
 2. **Maintain Consistency:** It ensures that different versions of software components (code, documentation, etc.) are consistent and reliable.
 3. **Team Collaboration:** SCM enables multiple developers or teams to work collaboratively on the same project without conflicts, through controlled processes.

Importance of SCM:

- **Consistency:** SCM guarantees that every version of the software, as well as its configuration items (CIs), are tracked and managed.
 - **Collaboration:** It enables teams (especially distributed ones) to work together by providing a unified view of software components and their history.
 - **Risk Reduction:** Helps in reducing risks related to configuration drift, where different parts of the system evolve without tracking.
-

2. SCM Key Concepts

2.1 Configuration Items (CIs)

- **Definition:** Configuration items are the artifacts, components, and documents that are part of the software system. These items include source code, design documents, libraries, test cases, executables, and even environments or configurations.
- **Examples:**
 - **Source Code:** Files such as `.cpp` or `.py` containing the program logic.
 - **Documentation:** Design specifications, user manuals, and test documentation.
 - **Build Scripts:** Files used to automate the process of building and testing the software.

2.2 Version Control

- **Definition:** Version control is the process of managing changes to configuration items, ensuring that versions are tracked, and history is preserved.
- **Importance:** It allows developers to collaborate, revert to previous versions, and manage updates without risking the integrity of the codebase.
- **Example:** Using Git to commit changes, branch new features, and merge changes into the main codebase.

2.3 Build Management

- **Definition:** Build management involves automating the software build process (compiling code, packaging it, etc.) to ensure consistency and reliability.
- **Importance:** It helps to ensure that every developer or automated system can consistently recreate the build environment.
- **Example:** Using Jenkins to trigger automated builds every time a change is pushed to the Git repository.

2.4 Change Control

- **Definition:** Change control refers to the formal process of requesting, reviewing, approving, and implementing changes to software.
 - **Importance:** It ensures that changes are introduced in an organized, systematic way to prevent introducing errors or conflicts.
 - **Example:** Using a change request system to manage which code changes are implemented in production.
-

3. Version Control Systems (VCS)

What is a Version Control System?

- **Definition:** A version control system is a tool that helps software development teams manage changes to source code over time, keeping a history of changes and facilitating collaboration.
 - **Types of Version Control:**
 1. **Centralized Version Control (CVCS):** All configuration items are stored in a central repository. Developers check out files to work on and check them back in after changes.
 - **Example:** SVN (Subversion), CVS.
 2. **Distributed Version Control (DVCS):** Each developer has a complete local copy of the repository, allowing them to work offline and push changes to a central repository later.
 - **Example:** Git, Mercurial.
-

4. Popular Version Control Tools

4.1 Git

- **Overview:** Git is a distributed version control system that is widely used in modern software development. It allows multiple developers to work independently on different branches of a project.
- **Advantages:**
 - **Branching:** Developers can create separate branches for new features or bug fixes, making it easy to work on multiple aspects of a project simultaneously.
 - **Merging:** Git allows developers to merge their changes into the main codebase with minimal conflict.
 - **Speed:** Git is fast, even with large repositories.
- **Example:** Using GitHub to host and manage repositories, where each commit is tracked, and pull requests are reviewed before merging.

4.2 Subversion (SVN)

- **Overview:** SVN is a centralized version control system that keeps all project files in a central repository, and developers work on the files they check out from this repository.
 - **Advantages:**
 - **Simplicity:** SVN is relatively easy to set up and use.
 - **Atomic Commits:** SVN commits happen in a single atomic operation, making it easy to rollback or cancel changes.
 - **Example:** A development team working on a monolithic application using SVN for managing code changes.
-

5. SCM Activities

5.1 Configuration Identification

- **Definition:** Configuration identification is the process of identifying and defining the configuration items that will be managed in SCM.
- **Importance:** By identifying and tracking CIs, teams ensure that the right versions of software artifacts are used in development and deployment.
- **Example:** Identifying which version of the database schema or API documentation should be associated with the latest release of the product.

5.2 Configuration Control

- **Definition:** Configuration control involves controlling changes to configuration items to ensure that only authorized changes are implemented.
- **Importance:** Helps prevent unauthorized modifications, ensuring the integrity of the software.

- **Example:** Using Git's pull request system to review and approve changes before they are merged into the main branch.

5.3 Configuration Status Accounting

- **Definition:** This activity tracks the status and history of configuration items.
- **Importance:** It allows teams to know which versions of software components have been developed, tested, or deployed.
- **Example:** Using JIRA to track the progress and status of a specific feature or bug fix.

5.4 Configuration Audits

- **Definition:** Configuration audits verify that the software's configuration matches the desired state and that processes are being followed correctly.
 - **Importance:** Ensures that the system is in compliance with internal or external standards.
 - **Example:** Conducting an audit to ensure that the production environment matches the version of software that was tested.
-

6. Change Management

6.1 Request for Change (RFC)

- **Definition:** A formal proposal for a change, typically initiated by a stakeholder.
- **Example:** A developer requests adding a new feature, like a search bar, to the application interface.

6.2 Impact Analysis

- **Definition:** The process of evaluating the potential effects of a proposed change.
- **Importance:** Ensures that changes don't negatively affect existing functionality.
- **Example:** Analyzing how adding a new feature (search functionality) will affect the backend database and the user interface.

6.3 Change Approval

- **Definition:** The process of formally approving or rejecting a proposed change.
- **Example:** The project manager or change control board (CCB) reviews the impact analysis and approves the change.

6.4 Implementation and Testing

- **Definition:** The change is implemented and tested to ensure it meets requirements without introducing new issues.
- **Example:** After approval, a developer codes the new search feature, which is then tested to ensure no regression occurs.

6.5 Release Management

- **Definition:** The process of deploying new changes or releases to production.
 - **Example:** After thorough testing, the search feature is deployed to the live system.
-

7. Build Management

What is Build Management?

- **Definition:** Build management involves automating the process of building and testing the software to ensure consistency, efficiency, and reliability.
 - **Importance:** It ensures that the software can be consistently built and tested across different environments.
 - **Example:** Using Jenkins to trigger an automated build and run tests after a developer pushes a new commit to the repository.
-

8. SCM Challenges

1. **Managing Complex Configurations:**
 - With increasing system complexity, managing configuration items becomes more challenging.
 2. **Team Collaboration:**
 - Managing collaboration across multiple teams, especially in a distributed environment.
 3. **Tracking Dependencies:**
 - Software projects often have interdependencies that must be carefully managed.
 4. **Risk of Conflicts:**
 - Merging changes made by different team members may cause conflicts that require resolution.
-

9. SCM Best Practices

1. **Use Version Control:**
 - Always use version control systems to manage code changes.
 - **Example:** Commit frequently to track progress and create backups.
2. **Automate Builds:**
 - Set up automated build systems like Jenkins to ensure consistency.
 - **Example:** Trigger automatic tests every time code is committed to the repository.
3. **Implement a Branching Strategy:**

- Use clear branching strategies like Git Flow to manage features, releases, and hotfixes.
 - **Example:** Use feature branches for new development and master/main for stable releases.
4. **Track Issues and Changes:**
- Use issue-tracking tools like JIRA or Trello to manage bugs and features.
 - **Example:** Track tasks and bugs in JIRA and link them to specific commits in Git.
-

10. Conclusion

- **Key Takeaways:**
 - SCM helps manage changes, track software components, and ensure collaboration.
 - Tools like Git, Jenkins, and JIRA improve efficiency and reduce errors.
 - By following best practices in version control, build management, and change control, teams can deliver high-quality, reliable software.